



PRÉ-RENTRÉE 2026-2027

# Dossier individuel

**Ahmed BENHADJ SALEM - parcours candidat individuel**

---

5 séances de 2 heures - 10 heures

Python 3 - théorie, pratique, tests et projet

CONFIDENTIEL

---

# 1. Situation

---

Ahmed est candidat individuel. Il a déjà présenté une première partie sans réussite. Les bilans montrent que les connaissances théoriques sont étendues, mais que la programmation reste moins fiable que les autres domaines.

## 2. Synthèse des bilans

---

### Positionnement Première

- 18 questions traitées sur 18 ;
- représentation binaire, Web, réseaux et données en tables solides ;
- programmation : 55,6 %, priorité à rectifier ;
- erreurs sur la négation, le choix `for/while`, `len` et `None`.

### Positionnement de niveau Terminale

- six domaines solides sur sept ;
- erreur persistante sur un accumulateur : somme `1+2+3` confondue avec le nombre d'itérations ;
- algorithmique, types construits et architecture théoriquement disponibles.

## 3. Décision pédagogique

---

Le stage ne doit pas lui imposer une reprise lente de notions déjà connues. Le parcours vise :

- fiabilité du code ;
- tests et cas limites ;
- autonomie ;
- explication orale ;
- gestion du temps ;
- transformation d'une spécification en programme.

## 4. Plan d'action

| Séance | Objectif personnel                                          | Exigence supplémentaire     | Indicateur                    |
|--------|-------------------------------------------------------------|-----------------------------|-------------------------------|
| 1      | fiabiliser négations et conditions                          | tests de bornes             | aucune erreur logique         |
| 2      | distinguer <code>for/while</code> , sécuriser accumulateurs | invariant et variant        | somme correcte et justifiée   |
| 3      | contrats et tests                                           | tests écrits avant code     | cas limites couverts          |
| 4      | tables et mutabilité                                        | validation des données      | CSV robuste                   |
| 5      | projet autonome                                             | architecture sans squelette | code documenté et oral fluide |

## 5. Suivi

| Compétence    | Initial | S1 | S2 | S3 | S4 | S5 | Final |
|---------------|---------|----|----|----|----|----|-------|
| Logique       | 2       |    |    |    |    |    |       |
| Boucles       | 2       |    |    |    |    |    |       |
| Accumulateurs | 2       |    |    |    |    |    |       |
| Fonctions     | 3       |    |    |    |    |    |       |
| Tests         | 1       |    |    |    |    |    |       |
| Structures    | 3       |    |    |    |    |    |       |
| CSV           | 3       |    |    |    |    |    |       |
| Autonomie     | 2       |    |    |    |    |    |       |
| Oral          | 2       |    |    |    |    |    |       |

## 6. Conseils pour le parcours candidat individuel

---

- programmer en temps limité sur de petits sujets ;
- verbaliser la spécification avant de coder ;
- utiliser systématiquement des assertions ;
- relire les bornes de `range` ;
- distinguer la taille d'une structure et ses indices ;
- expliquer une fonction en termes d'entrées, sorties et cas limites ;
- conserver une preuve de chaque exercice exécuté.

## 7. Modèle de bilan final

---

### Plan de travail vers l'épreuve



PRÉ-RENTRÉE 2026-2027

# Fiabilisation ciblée

Ahmed BENHADJ SALEM - Python

---

5 séances de 2 heures - 10 heures

Python 3 - théorie, pratique, tests et projet

Document élève

---

## Logique et conditions

---

1. Nier `x > 5` puis `x >= 5`.
2. Nier `(x > 0) and (x < 10)`.
3. Écrire un jeu de tests sur les bornes.

## Choix de boucle

---

1. Justifier `for` pour parcourir un fichier déjà chargé.
2. Justifier `while` pour demander une saisie valide.
3. Donner un variant de boucle.

## Taille et indices

---

1. Pour une liste de longueur `n`, donner premier et dernier indice.
2. Écrire une boucle sûre utilisant les indices.
3. Corriger un accès `L[len(L)]`.

## Accumulateurs

---

1. Tracer la somme de `range(1,4)`.
2. Calculer simultanément somme et nombre de valeurs positives.
3. Expliquer l'invariant de somme.

## Fonctions et tests

---

1. Expliquer la différence `print` / `return` / `None`.
2. Écrire une fonction de moyenne avec contrat.
3. Écrire cinq tests, dont liste vide.
4. Lire `math.isclose` et tester `0.1+0.2`.

# Défi chronométré

---

En 15 minutes, écrire et tester une fonction qui renvoie le deuxième plus grand élément distinct d'une liste, ou `None` s'il n'existe pas.

# QCM blanc - Candidat individuel Première NSI

---

**Durée : 2 heures - 42 questions - 7 parties - calculatrice interdite.**

Une seule réponse est correcte. Le format d'entraînement reprend l'évaluation ponctuelle officielle : 1 point par bonne réponse, 0 sinon, puis conversion sur 20 par  $\text{points} \times 20 / 42$ .

**Différence avec le test de positionnement Nexus :** dans ce QCM blanc, une réponse fausse n'enlève pas de point. Après une première passe prudente, il est rationnel de répondre à toutes les questions.

## Partie - Données de base

---

### Question 1

101101 en binaire vaut en décimal :

- ☐ A. 35
- ☐ B. 45
- ☐ C. 46
- ☐ D. 53

### Question 2

Sur 8 bits non signés, la plus grande valeur est :

- ☐ A. 127
- ☐ B. 128
- ☐ C. 255
- ☐ D. 256



## Question 3

En complément à 2 sur 8 bits, -1 est représenté par :

- ☐ A. 00000001
- ☐ B. 10000001
- ☐ C. 11111111
- ☐ D. 10000000

## Question 4

En Python, l'affirmation correcte est :

- ☐ A.  $0.1+0.2$  vaut toujours exactement 0.3
- ☐ B. les flottants sont des rationnels exacts
- ☐ C. il faut éviter de tester directement certaines égalités de flottants
- ☐ D. float et int ont toujours la même représentation

## Question 5

`True and not False` vaut :

- ☐ A. True
- ☐ B. False
- ☐ C. None
- ☐ D. 1.0

## Question 6

Unicode sert principalement à :

- ☐ A. compresser les images
- ☐ B. identifier des caractères de nombreuses écritures
- ☐ C. chiffrer les fichiers
- ☐ D. router des paquets

# Partie - Types construits

---

## Question 7

Un tuple Python est :

- ☐ A. toujours modifiable
- ☐ B. un regroupement ordonné non mutable
- ☐ C. un dictionnaire
- ☐ D. un fichier CSV

## Question 8

Pour `L=[10,20,30]` , `L[2]` vaut :

- ☐ A. 20
- ☐ B. 30
- ☐ C. IndexError
- ☐ D. 3

## Question 9

`[x*x for x in range(4)]` vaut :

- ☐ A. [1,4,9,16]
- ☐ B. [0,1,4,9]
- ☐ C. [0,1,2,3]
- ☐ D. [4]

## Question 10

Pour `M=[[1,2],[3,4]]` , `M[1][0]` vaut :

- ☐ A. 1
- ☐ B. 2
- ☐ C. 3
- ☐ D. 4

## Question 11

Pour `d={"a":1}`, ajouter la clé `b` associée à 2 se fait par :

- ☐ A. `d.add("b",2)`
- ☐ B. `d["b"]=2`
- ☐ C. `d.append(2)`
- ☐ D. `d("b")=2`

## Question 12

Après `a=[1,2]` ; `b=a` ; `b.append(3)`, `a` vaut :

- ☐ A. `[1,2]`
- ☐ B. `[1,2,3]`
- ☐ C. `[3]`
- ☐ D. erreur

# Partie - Tables

---

## Question 13

Dans une table CSV, une ligne correspond généralement à :

- ☐ A. un descripteur
- ☐ B. un enregistrement
- ☐ C. un fichier entier
- ☐ D. un type Python

## Question 14

`csv.DictReader` produit chaque ligne sous forme de :

- ☐ A. chaîne unique
- ☐ B. liste d'entiers
- ☐ C. dictionnaire
- ☐ D. tuple de bytes

## Question 15

Pour garder les lignes de score  $\geq 10$ , on effectue :

- ☐ A. une projection
- ☐ B. un filtrage
- ☐ C. une fusion
- ☐ D. un encodage

## Question 16

Trier une liste de dictionnaires `t` par score peut s'écrire :

- ☐ A. `sorted(t, key=lambda x: x["score"])`
- ☐ B. `t.sort("score")`
- ☐ C. `sort(score,t)`
- ☐ D. `sorted("score")`

## Question 17

Fusionner deux tables suppose notamment :

- ☐ A. aucun champ commun
- ☐ B. un critère de correspondance
- ☐ C. qu'elles aient la même longueur
- ☐ D. qu'elles soient triées en binaire

## Question 18

Une métadonnée est :

- ☐ A. une donnée fausse
- ☐ B. une donnée qui décrit une autre donnée
- ☐ C. une donnée secrète
- ☐ D. une copie de sauvegarde

# Partie - Web

---

## Question 19

Le rôle principal de HTML est de :

- ☐ A. mettre en forme
- ☐ B. structurer le contenu
- ☐ C. router les paquets
- ☐ D. stocker les utilisateurs

## Question 20

Le rôle principal de CSS est de :

- ☐ A. présenter le contenu
- ☐ B. exécuter Python
- ☐ C. définir les adresses IP
- ☐ D. gérer le disque

## Question 21

Dans le modèle client-serveur :

- ☐ A. le serveur interroge toujours le client
- ☐ B. le client envoie une requête et reçoit une réponse
- ☐ C. aucun protocole n'est utilisé
- ☐ D. le navigateur fabrique tout sans réseau

## Question 22

Une requête GET est adaptée lorsque :

- ☐ A. on transmet un mot de passe en clair dans l'URL
- ☐ B. on demande une ressource sans effet sensible
- ☐ C. on veut garantir la confidentialité
- ☐ D. on remplace HTTPS

## Question 23

HTTPS apporte principalement :

- ☐ A. la compression
- ☐ B. le chiffrement et l'authentification du serveur
- ☐ C. un stockage local
- ☐ D. une adresse IP fixe

## Question 24

Un gestionnaire d'événement est exécuté :

- ☐ A. lorsqu'un événement associé se produit
- ☐ B. uniquement au démarrage du serveur
- ☐ C. avant le chargement de la page dans tous les cas
- ☐ D. par le routeur

# Partie - Architecture

---

## Question 25

Dans une architecture de von Neumann, le processeur :

- ☐ A. stocke uniquement les images
- ☐ B. exécute les instructions
- ☐ C. remplace le système d'exploitation
- ☐ D. attribue les adresses IP

## Question 26

Le système d'exploitation :

- ☐ A. gère ressources, fichiers et processus
- ☐ B. est une page Web
- ☐ C. est un protocole de routage
- ☐ D. remplace la mémoire vive

## Question 27

Une adresse IP sert à :

- ☐ A. identifier une machine sur un réseau
- ☐ B. nommer un fichier
- ☐ C. compresser un paquet
- ☐ D. chiffrer une chaîne

## Question 28

Un routeur :

- ☐ A. exécute les fonctions Python
- ☐ B. transmet les paquets vers leur destination
- ☐ C. convertit toujours HTML en CSS
- ☐ D. stocke uniquement les mots de passe

## Question 29

Le découpage en paquets permet notamment :

- ☐ A. d'interdire tout partage du réseau
- ☐ B. de transmettre et réassembler des données
- ☐ C. de supprimer les protocoles
- ☐ D. de garantir zéro perte sans mécanisme

## Question 30

Sous Unix, la commande `ls` sert à :

- ☐ A. lister les fichiers
- ☐ B. changer de répertoire
- ☐ C. supprimer un fichier
- ☐ D. afficher une adresse IP

# Partie - Programmation

---

## Question 31

Après `a=3` ; `b=a` ; `a=5` , b vaut :

- ☐ A. 3
- ☐ B. 5
- ☐ C. 8
- ☐ D. None

## Question 32

La négation de `x > 5` est :

- ☐ A. `x < 5`
- ☐ B. `x >= 5`
- ☐ C. `x <= 5`
- ☐ D. `x != 5`

## Question 33

`range(2,8,2)` produit :

- ☐ A. 2,4,6
- ☐ B. 2,4,6,8
- ☐ C. 2,3,4,5,6,7
- ☐ D. 8,6,4,2

## Question 34

Une boucle while est particulièrement adaptée lorsque :

- ☐ A. le nombre de tours dépend d'une condition
- ☐ B. le nombre de tours est toujours exactement 3
- ☐ C. on veut définir un dictionnaire
- ☐ D. on écrit une page HTML



## Question 35

Une fonction sans return exécuté renvoie :

- ☐ A. 0
- ☐ B. False
- ☐ C. None
- ☐ D. une erreur systématique

## Question 36

Un bon jeu de tests :

- ☐ A. prouve toujours la correction
- ☐ B. comprend cas normaux et limites
- ☐ C. ne contient qu'un exemple
- ☐ D. remplace la spécification

# Partie - Algorithmique

---

## Question 37

La recherche séquentielle :

- ☐ A. examine des éléments un à un
- ☐ B. exige une liste triée
- ☐ C. est toujours logarithmique
- ☐ D. trie la liste

## Question 38

Pour calculer un maximum dans une liste non vide, une bonne initialisation est :

- ☐ A. 0 dans tous les cas
- ☐ B. le premier élément
- ☐ C. None sans traitement
- ☐ D. la longueur de la liste

## Question 39

Le tri par sélection place à chaque étape :

- ☐ A. un élément choisi au hasard
- ☐ B. le plus petit élément restant à sa position
- ☐ C. la moitié droite à gauche
- ☐ D. la moyenne en tête

## Question 40

Dans le pire cas, les tris insertion et sélection étudiés sont de coût :

- ☐ A. constant
- ☐ B. logarithmique
- ☐ C. linéaire
- ☐ D. quadratique

## Question 41

La recherche dichotomique nécessite :

- ☐ A. une liste triée
- ☐ B. un dictionnaire vide
- ☐ C. un réseau
- ☐ D. une fonction récursive obligatoirement

## Question 42

Une stratégie gloutonne :

- ☐ A. explore forcément toutes les solutions
- ☐ B. fait un choix local à chaque étape
- ☐ C. garantit toujours l'optimum pour tout problème
- ☐ D. est identique à la dichotomie